

第6章 面向对象

建议学时:5-7 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握类与对象的声明、构造器、this 与字段初始化
- 掌握封装与四种访问修饰符,理解"信息隐藏"的工程意义
- 掌握继承 extends、super、方法重写与 @Override 注解
- 理解多态与动态绑定,与 C 的虚函数表对照
- 掌握抽象类与接口的区别与选择,会写接口默认方法
- 掌握 equals/hashCode 契约:为什么必须同时重写、怎么写才对
- 掌握内部类、匿名类与 static 嵌套类的使用场景

💡 从 C 到 Java:面向对象的动机

C 程序员其实一直在写"半吊子面向对象":结构体 = 类字段; `shape_area(s)` 函数 = 方法; `void*` + 函数指针表 = 多态。Java 把这套手工活变成语言级设施:数据与操作它的函数被强制绑在一起(封装),类型体系显式表达"是什么/能干什么"(继承与接口),运行时按实际类型选方法(多态)。

继承在 C 里是"结构体嵌套 + 指针强转"——Linux 内核的 `list_head` 就是这么玩的,但类型不安全、容易错。Java 的 `extends` 让"子类是父类的一种"成为编译器检查的规则,漏了转型错误直接编译失败。

接口是 C 程序员最陌生的概念。它像"纯函数指针表 + 契约"的组合:类通过 `implements` 承诺"我能做这些事",调用方只依赖接口不依赖具体类——这是插件、框架、测试替身(mock)的地基。C 里你手工写 `struct ops { int (*area)(void*); }`,Java 里就是 `interface Shape { double area(); }`。

equals/hashCode 是重灾区。C 里比较结构体用 `memcmp`(有坑:padding 字节);Java 里 `==` 比较引用、`equals` 比较内容——但"内容相同"由你定义。写错 `hashCode` 的直接后果:HashMap/HashSet 里的元素找不到。这是面试与生产事故的高发区,\$6.6 给出唯一正确的写法模板。

学习顺序:\$6.1-6.4 建立"类/继承/多态"三板斧,\$6.5 接口,\$6.6 equals/hashCode 契约(必考必用),\$6.7-6.8 补充形态。

📦 前置知识自查

- C 结构体、函数指针、`void*` 类型擦除式多态
- 第1章:类型与数组;第4章:方法;第5章:对象在堆上的生命周期
- 第2章:`==` 与 `equals` 的区别将在本章展开

🚀 本章学习路线

1. **第一阶段(约 2 小时):**\$6.1-6.4,把 C 的 struct+函数指针 逐个翻译成 Java 类,体会"语言设施 vs 手工约定"
2. **第二阶段(约 1.5 小时):**\$6.5 接口与 \$6.7 匿名类,理解"面向接口编程"
3. **第三阶段(约 1.5 小时):**\$6.6 equals/hashCode 契约反复抄写模板,再完成章末实验
4. **验收标准:**能讲清"封装=让状态只被自己的方法修改";能写出放 HashMap 的键类;能说出接口与抽象类的选择依据

本章知识导图

- §6.1 类与对象:声明 / 构造器 / this
- §6.2 封装与访问控制:private / public / protected
- §6.3 继承:extends / super / 重写 / final
- §6.4 多态与动态绑定:与 C 虚函数对照
- §6.5 抽象类与接口:默认方法 / 选择依据
- §6.6 equals / hashCode 契约
- §6.7 内部类 / 匿名类 / static 嵌套类
- §6.8 static 成员与初始化顺序

§6.1 类与对象

6.1.1 从结构体到类

C 结构体	Java 类
struct Point { int x, y; };	class Point { int x, y; }
Point p = {1, 2};(栈上)	Point p = new Point();(堆上,字段默认 0/null)
函数:double dist(Point* p)	方法:double dist()(直接访问自己的字段)
初始化靠手写或 memset	构造器保证"对象一出生就合法"

示例 6-1 类的基本形态

```

public class PointDemo {
    static class Point {
        int x;           // 字段:默认 0
        int y;           // 字段:默认 0

        // 构造器:与类同名,没有返回类型,new 时自动执行
        Point(int x, int y) {
            this.x = x;       // this 区分"参数 x"与"字段 x"
            this.y = y;
        }

        // 无参构造器(重载):如果没写任何构造器,编译器会送一个默认的
        Point() {
            this(0, 0);       // 调用另一个构造器(必须是第一句)
        }

        double distance() {   // 方法:能直接摸自己的字段
            return Math.sqrt((double) x * x + y * y);
        }

        @Override
        public String toString() { // 第11章讲注解;这里标记"重写"
            return "(" + x + ", " + y + ")";
        }
    }

    public static void main(String[] args) {
        Point p = new Point(3, 4); // new = 分配堆内存 + 调构造器
        Point q = new Point();     // 用无参构造器
        System.out.println(p);     // (3, 4):println 自动调 toString
        System.out.println(p.distance()); // 5.0
        System.out.println(q);     // (0, 0)
    }
}

```

⚠ 易错点 1:忘了 new 与字段默认值

C 里 `Point p;` 是"申请栈空间,内容未定义";Java 里局部变量不初始化直接读会 编译报错 (比 C 安全);但字段(成员变量)不写也有默认值:int 0、double 0.0、boolean false、引用 null。"字段有默认值、局部变量没有"是新手最容易错记的规则。另外 `new Point()` 是"堆上造对象",`Point p;` 只是声明一个引用变量,不调用构造器。

§6.2 封装与访问控制

6.2.1 四种访问级别

修饰符	同类	同包	子类	任意处	C 的对应概念
private	✓				static 函数里的内部状态(头文件里藏起来)
(无,包私有)	✓	✓			同 .c 文件内的静态符号
protected	✓	✓	✓		"继承者可见"的半公开 API
public	✓	✓	✓	✓	头文件里导出的接口

示例 6-2 封装:状态只由方法修改

```

public class BankAccountDemo {
    static class BankAccount {
        private double balance;    // 私有:外面碰不到

        BankAccount(double init) {
            if (init < 0) throw new IllegalArgumentException("初始金额不能为负");
            this.balance = init;
        }

        public void deposit(double amount) {    // 公开接口
            if (amount <= 0) throw new IllegalArgumentException("金额必须为正");
            balance += amount;
        }

        // 取款:内部校验余额,这是"业务规则"放在方法里的意义
        public boolean withdraw(double amount) {
            if (amount <= 0) throw new IllegalArgumentException("金额必须为正");
            if (balance < amount) return false;    // 余额不足,拒绝
            balance -= amount;
            return true;
        }

        public double getBalance() {    // 只读访问器(getter)
            return balance;
        }
    }

    public static void main(String[] args) {
        BankAccount acc = new BankAccount(100);
        acc.deposit(50);
        acc.withdraw(30);
        System.out.println("余额: " + acc.getBalance());    // 120.0
        // acc.balance = 999999;    // 编译错误:balance 是 private
    }
}

```

★ 重点:封装的工程意义

封装不是"不让别人用",是 让状态的变化只能经过"你的规则"。C 里 `acc.balance = -5;` 随手就写;Java 里 `balance` 私有后,唯一的入口是带校验的 `deposit/withdraw`,非法状态在编译期就被挡在门外。生产要点:字段全 `private`,方法按需 `public`;需要只读暴露用 `getter`,不要直接开字段。校验失败用异常(第9章)而非静默容忍。

§6.3 继承:extends 与 super

6.3.1 继承 = is-a + 代码复用

子类 `extends` 父类:自动拥有父类的非私有字段与方法,可重写(override)方法改变行为, `super` 调用父类版本。Java 单继承(一个类只有一个父类)。**构造器链:**`new` 子类时,先从父类构造器开始(隐式调 `super()`),再到子类。

示例 6-3 继承与重写

```

public class InheritDemo {
    static class Animal {
        protected String name;           // protected:子类可见

        Animal(String name) { this.name = name; }

        public void speak() {
            System.out.println(name + " 发出声音");
        }
    }

    static class Dog extends Animal {      // 继承
        Dog(String name) { super(name); } // 必须第一句调用父类构造器

        @Override                         // 注解:让编译器检查"真的重写了"
        public void speak() {
            System.out.println(name + " 汪汪叫");
        }

        public void fetch() {              // 子类新增能力
            System.out.println(name + " 捡球");
        }
    }

    public static void main(String[] args) {
        Dog d = new Dog("旺财");
        d.speak();                         // 旺财 汪汪叫(重写生效)
        d.fetch();                         // 旺财 捡球(子类独有)

        Animal a = new Dog("大黄");        // 子类对象赋给父类引用:向上转型
        a.speak();                         // 大黄 汪汪叫(动态绑定,见 §6.4)
        // a.fetch();                      // 编译错误:Animal 没有 fetch(静态类型说了算)
    }
}

```

⚠ 易错点 2:重写与重载是两回事

重写(override):子类方法签名与父类完全一致,行为替换;加 `@Override` 让编译器把关(签名写错会编译报错)。**重载(overload)**:同一类里同名不同参(第4章)。C 没有这两个概念——重写在 C 里靠函数指针表替换,重载靠名字不同(`printf_float`之类的命名)。**重写看对象类型,重载看引用类型**,这个区分在第4章与本章的交叉点高频出错。

§6.4 多态与动态绑定

6.4.1 C 的虚函数表 vs Java 的动态绑定

C 的面向对象多态: `struct ShapeOps { double (*area)(void*); }`,每个形状结构体里放自己的函数指针表,调用 `ops->area(shape)` 时按"表里存的函数"分发。Java 做的是一模一样的事,由 JVM 自动完成:方法调用时,按对象的实际类型(不是引用类型)查找方法并执行——这就是动态绑定(多态)。

示例 6-4 多态:一个接口,多种实现

```

public class PolyDemo {
    static abstract class Shape {           // 抽象类($6.5):不能 new
        abstract double area();           // 抽象方法:子类必须实现
    }

    static class Circle extends Shape {
        final double r;
        Circle(double r) { this.r = r; }
        double area() { return Math.PI * r * r; }
    }

    static class Rect extends Shape {
        final double w, h;
        Rect(double w, double h) { this.w = w; this.h = h; }
        double area() { return w * h; }
    }

    public static void main(String[] args) {
        Shape[] shapes = {new Circle(1), new Rect(2, 3), new Circle(2)};
        double total = 0;
        for (Shape s : shapes) {
            total += s.area();           // 运行时按实际类型分发!
        }
        System.out.printf("总面积 = %.2f\n", total);    // 3.14+6+12.57 = 21.71
        // C 的写法:shapes[i].ops->area(shapes[i]);Java 自动完成
    }
}

```

★ 重点:多态解决什么问题

多态让"新增一种形状"不需要修改汇总代码:再写一个 `Triangle extends Shape`,上面循环零改动就能算面积。开闭原则:对扩展开放,对修改关闭——这是框架、插件、测试替身的技术根基。生产中的日常形态:接口定义契约,多个实现类,调用方只依赖接口(面向接口编程)。

§6.5 抽象类与接口

6.5.1 两者的区别与选择

对比项	抽象类	接口
关键词	abstract class	interface
继承方式	extends(单继承)	implements(可多实现)
字段	可有实例字段	只能有常量(public static final)
构造器	有(子类走 super)	没有
方法	抽象方法 + 普通方法	抽象方法 + 默认方法(default) + 静态方法
语义	"is-a" 且共享状态/逻辑	"能做什么"的契约
C 的对应	结构体 + 部分实现的函数指针	纯函数指针表(ops 结构)

示例 6-5 接口与默认方法

```

public class InterfaceDemo {
    // 接口:定义"能做什么"
    interface Sortable {
        int size();

        boolean less(int i, int j);

        void swap(int i, int j);

        // 默认方法:JDK8+,给接口补"附带实现"的能力
        default void sort() { // 冒泡排序,直接可复用
            for (int i = 0; i < size() - 1; i++) {
                for (int j = 0; j < size() - 1 - i; j++) {
                    if (less(j + 1, j)) {
                        swap(j, j + 1);
                    }
                }
            }
        }
    }
}

// 实现接口:只需三个抽象方法,sort 白送
static class IntArray implements Sortable {
    private final int[] data;

    IntArray(int[] data) { this.data = data; }

    public int size() { return data.length; }

    public boolean less(int i, int j) { return data[i] < data[j]; }

    public void swap(int i, int j) {
        int t = data[i];
        data[i] = data[j];
        data[j] = t;
    }
}

public static void main(String[] args) {
    IntArray arr = new IntArray(new int[]{5, 3, 8, 1, 9, 2});
    arr.sort(); // 默认方法:一行排序
    System.out.println(java.util.Arrays.toString(arr.data)); // [1, 2, 3, 5, 8, 9]
}
}

```

⚠ 易错点 3:接口 vs 抽象类的选择

选择口诀: "要共享状态、且语义是 is-a" → 抽象类; "定义契约、允许多角色" → 接口。典型错误:为了复用一段代码就继承一个不相干的类(比如让 Account 继承 ArrayList)——继承是"is-a"关系,不是"拿来用"。Java 类单继承、接口可多实现,接口是组合式的"能力声明",抽象类才是"家族血统"。

§6.6 equals 与 hashCode 契约

6.6.1 为什么必须同时重写

`==` 比较引用是否同一个;Object 的默认 `equals` 就是 `==`。想让"内容相同"成立,重写 `equals`。而 `HashMap/HashSet` 靠 `hashCode` 分桶:两个对象 `equals` 为 `true`,则 `hashCode` 必须相等;`hashCode` 相等不要求 `equals` (碰撞)。只重写 `equals` 不重写 `hashCode`: `map.put(k1, v); map.get(k2) (k1.equals(k2))` 返回 `null`——因为 `k2` 哈希到了另一个桶。这就是契约:违反的代价是集合失灵。

示例 6-6 equals/hashCode 标准模板(JDK7+ 写法)

```

import java.util.HashMap;
import java.util.Map;
import java.util.Objects;

public class EqualsDemo {
    static class Student {
        private final String id;        // 学号:判等依据
        private final String name;

        Student(String id, String name) {
            this.id = id;
            this.name = name;
        }

        // 模板步骤:① 自反 ② 类型判断 ③ 字段逐个比
        @Override
        public boolean equals(Object o) {
            if (this == o) return true;           // 同引用,直接相等
            if (!(o instanceof Student)) return false; // 类型不符(含 null)
            Student s = (Student) o;
            return id.equals(s.id) && name.equals(s.name);
        }

        // 与 equals 用同一组字段
        @Override
        public int hashCode() {
            return Objects.hash(id, name); // JDK7+:一行生成
        }

        @Override
        public String toString() {
            return id + ":" + name;
        }
    }

    public static void main(String[] args) {
        Map<Student, String> grades = new HashMap<>();
        Student s1 = new Student("2024001", "张三");
        Student s2 = new Student("2024001", "张三"); // 内容相同,引用不同
        grades.put(s1, "A");

        System.out.println(s1 == s2);           // false:引用不同
        System.out.println(s1.equals(s2));       // true:内容相同
        System.out.println(grades.get(s2));      // A:靠 hashCode 找到同桶 + equals 命中

        // 反例:只改 equals 不改 hashCode 会导致 get 返回 null
    }
}

```

⚠ 易错点 4:equals 的对称性与可变字段

两条生产铁律: ① equals/hashCode 依据的字段必须是"不可变"的 (final 或从不修改)——字段变了,对象的哈希就变了,放进 HashSet 后再改字段,元素会"失踪"; ② 继承场景重写 equals 要小心对称性:父类 p.equals(子类 s) 为 true 但 s.equals(p) 为 false 会破坏契约——生产建议:需要子类判等时,用"字段比较"而非 instanceof,或干脆组合优先于继承。

§6.7 内部类、匿名类与静态嵌套类

6.7.1 三种形态与用途

- **静态嵌套类**: static class Inner, 不持有外部类引用,当"把相关类型聚在一起"用(第4章实验里的 Point 就是)
- **内部类**: 普通内部类,隐式持有外部类引用,可访问外部私有成员;常用于"服务外部类的小助手"
- **匿名类**: new 接口() { 实现 }, 一次性实现;在 lambda 出现后,单方法接口优先用 lambda

示例 6-7 匿名类与 lambda 的等价替换

```
import java.util.Arrays;
import java.util.Comparator;

public class AnonymousDemo {
    public static void main(String[] args) {
        String[] names = {"Tom", "Alice", "Bob"};

        // 匿名类:JDK8 之前唯一的"内联实现"方式
        Arrays.sort(names, new Comparator<String>() {
            @Override
            public int compare(String a, String b) {
                return a.length() - b.length();
            }
        });

        // lambda:完全等价,现代写法
        Arrays.sort(names, (a, b) -> a.length() - b.length());

        // 方法引用:最短
        Arrays.sort(names, Comparator.comparingInt(String::length));

        System.out.println(Arrays.toString(names));    // [Tom, Bob, Alice]
    }
}
```



工程建议:匿名类的去留

新代码里,函数式接口一律用 lambda;匿名类只留给"多个方法的接口需要一次性实现"的场景。匿名类持有外部引用,可能造成意外捕获(比如在回调里误用了可变外部变量)——lambda 捕获 effectively final 变量(\$4.5),匿名类同样受限,但写法上更容易犯错。看到旧代码里的匿名类,能读、能改写成 lambda 即可。

§6.8 static 成员与初始化顺序

6.8.1 static = 属于类,不属于对象

static 字段:所有实例共享一份(相当于 C 的全局变量,但归属类); **static 方法**:不依赖实例(工具方法,第4章); **static 初始化块**:类第一次加载时执行一次。初始化顺序:父类 static → 子类 static → 父类字段/构造 → 子类字段/构造。用 static 字段做计数器是经典用法。

示例 6-8 静态计数器与初始化顺序

```

public class StaticDemo {
    static class Employee {
        private static int nextId = 1;    // 类级:所有实例共享
        private final int id;             // final 字段:构造时定死

        static {                          // 静态块:类加载时执行一次
            System.out.println("[静态块] Employee 类加载");
        }

        Employee() {
            this.id = nextId++;            // 每 new 一个,编号递增
        }

        int getId() { return id; }
    }

    public static void main(String[] args) {
        Employee e1 = new Employee();
        Employee e2 = new Employee();
        Employee e3 = new Employee();
        System.out.println(e1.getId() + "," + e2.getId() + "," + e3.getId());
        // 1,2,3:编号是类级计数器分配的,实例间共享
        System.out.println(Employee.nextId); // 4:static 字段可直接类名访问
    }
}
// 输出:[静态块] Employee 类加载 只出现一次—一类只加载一次

```

⚠ 易错点 5:static 方法里不能碰实例成员

`static` 方法没有 `this`——在 `static` 方法里访问实例字段/实例方法,编译报错。C 的对等直觉:static 方法 = "不传 `this` 的全局函数"。main 是 `static` 的,所以所有代码都在 `static` 上下文里开场,想用实例成员必须 `new` 出来。另一个坑:static 字段是全局的,多线程并发读写要加锁(第9章)。

本章速查表

主题	要点
类 vs 结构体	类=字段+方法+构造器;new 在堆上造对象;字段有默认值,局部变量没有
构造器	与类同名无返回;this(...) 链式调用必须第一句;不写构造器有默认无参版
访问控制	private 最严 → 包私有 → protected(子类可见)→ public;字段一律 private
封装	状态只经方法修改;getter 只读暴露;非法状态在方法内拦截
继承	extends 单继承;构造器链先父后子;super 调父类;final 类不可继承
重写	签名相同、行为替换,必须加 @Override;与重载(同名不同参)区分
多态	按实际类型动态绑定;C 的虚函数表自动版;开闭原则
抽象类	abstract class;可含字段与实现;is-a 共享状态时用
接口	implements 可多实现;纯契约;default 方法 JDK8+
equals/hashCode	equals 相同 → hashCode 必须相同;模板:this==o / instanceof / 字段比;Objects.hash 生成
判等字段	必须不可变;变了会让 HashSet 元素"失踪"
三种嵌套类	static 嵌套(无外部引用)/ 内部类(有外部引用)/ 匿名类(一次性;单方法用 lambda)
static 成员	属于类;static 方法无 this 不能用实例成员;初始化顺序:父 static → 子 static → 构造

最小必会清单

- 能写出含构造器、this、getter 的类,并说出字段默认值与局部变量的区别
- 能说出四种访问修饰符的可见范围与 C 的对应
- 能写出继承 + super + @Override 重写的代码,并解释构造器链顺序
- 能解释多态与动态绑定,说出它与 C 虚函数表的关系
- 能说出抽象类与接口的选择依据,会写接口 default 方法
- 能默写 equals/hashCode 标准模板并解释契约
- 能说出三种嵌套类的区别,会把匿名类改写为 lambda

自测题

Q 1. 下列哪些是合法重写(有 @Override)?父类 `void f(int x)`:① `void f(int x)`;② `int f(int x)`;③ `void f(long x)`;④ `void f(int x) throws Exception`

✓ 参考答案

只有 ① 合法。② 返回类型不同且不是协变,编译错误;③ 参数不同是重载不是重写(父类方法仍存在);④ 重写方法不能抛比父类更宽泛的 checked 异常——Exception 比父类的(无)宽,编译错误。

Q 2. 为什么 HashMap 的键类必须同时重写 equals 和 hashCode?只重写 equals 会怎样?

✓ 参考答案

HashMap 先用 hashCode 定位桶,再用 equals 在桶内比较。两个“内容相同”的键,若 hashCode 不同就落入不同桶,get 用第二个键查找时找不到第一个键存入的值。契约:equals 相同 → hashCode 必须相同。所以必须用同一组字段同时实现两者。

Q 3. 一个 Student 类以学号 id 为判等字段。把两个内容相同的 Student 放入 HashSet 后,再调用 set 里那个对象的 setId("2024999"),会发生什么?

✓ 参考答案

元素“失踪”:hashCode 依赖 id,id 变了哈希变,但对象在 HashSet 里还是按旧哈希所在桶存放。contains/remove 用新哈希查找,找不到。这就是“判等字段必须不可变”的原因——这也是为什么生产代码的 id 通常 final。

Q 4. 接口与抽象类各自适合什么场景?举一个“该用接口却用了抽象类”的反例。

✓ 参考答案

接口:定义“能做什么”的契约,允许多角色(如 Runnable、Comparable);抽象类:is-a 关系且要共享状态与实现。反例:让 Cat extends Dog 只为复用 eat()——没有血缘关系却继承,应改成 interface Eatable + 组合。判断标准:问“子类是父类的一种吗?”,不是就绝不继承。

Q 5. 写出结果:父类构造器打印“A”,子类构造器打印“B”,new Child() 输出什么?如果父类只有带参构造器呢?

✓ 参考答案

输出“A”然后“B”:子类构造器隐式调用 super(),先执行父类构造器。若父类没有无参构造器,子类必须显式 super(参数)且必须是构造器第一句,否则编译错误。C 的对应:结构体嵌套时先初始化内层成员。

Q 6. 为什么生产代码里字段要 private?直接 public 不更简单吗?

✅ 参考答案

public 字段把"状态"与"规则"分离:任何代码都能改成非法值(负余额、空名称)。private + 方法入口,让所有修改经过校验逻辑;同时改内部实现(比如 balance 从 double 改 BigDecimal)不影响调用方。这正是 C 里"头文件只暴露函数,结构体定义藏在 .c 里"的同一门手艺。

章末实验题:教务管理系统

📄 实验 6:教务管理系统

实验目标:编写一个小型教务管理系统,综合运用本章全部知识点:类与构造器、封装、继承、多态、接口、equals/hashCode、匿名类/lambda。

实验要求:

- 任务 1:定义 `Person` 类:私有 name/age 字段 + 构造器 + getter + toString(覆盖 §6.1-6.2 类与封装)
- 任务 2:定义 `Student extends Person`:新增学号 id,重写 toString;定义 `Teacher extends Person`:新增课程 course(覆盖 §6.3 继承)
- 任务 3:定义接口 `Assessable` (能考核): `double score()`;Student 按成绩、Teacher 按课时返回(覆盖 §6.4-6.5 多态与接口)
- 任务 4:给 Student 按学号实现 equals/hashCode,用 HashSet 去重:同号学生只留一个(覆盖 §6.6 契约)
- 任务 5:用 lambda 按成绩降序排序并打印前 3 名(覆盖 §6.7 函数式与比较器)
- 任务 6:定义静态计数器统计创建的学生总数(覆盖 §6.8 static)
- 任务 7:main 输出完整报告:人员名单、去重结果、排名(覆盖全部)

实验环境:JDK 17+; javac SchoolSystem.java 编译运行。

第一步 写 Person/Student/Teacher 骨架,编译通过后再加接口与判等

第二步 任务 4 验证去重:put 两个同号不同名对象,set.size() 应为 1

第三步 任务 5 与第4章的 Stream 结合:用 stream().sorted(...) 完成排名

✅ 参考答案要点

```

import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Objects;
import java.util.Set;

/** 教务管理系统 — 第6章综合实验参考答案
 * 覆盖:类/封装、继承、多态、接口、equals/hashCode、lambda、static
 */
public class SchoolSystem {

    // 任务 1:基类,封装 name/age
    static class Person {
        private final String name;
        private final int age;

        Person(String name, int age) {
            this.name = name;
            this.age = age;
        }

        public String getName() { return name; }

        public int getAge() { return age; }

        @Override
        public String toString() { return name + "(" + age + ")"; }
    }

    // 任务 2:学生,继承 Person,新增学号
    static class Student extends Person {
        private final String id;
        private final double examScore;    // 期末成绩

        Student(String id, String name, int age, double examScore) {
            super(name, age);    // 先父后子
            this.id = id;
            this.examScore = examScore;
        }

        public String getId() { return id; }

        @Override
        public String toString() { return id + ":" + super.toString(); }

        // 任务 4:按学号判等(判等字段 final,永不改变)
        @Override
        public boolean equals(Object o) {
            if (this == o) return true;
            if (!(o instanceof Student)) return false;
            Student s = (Student) o;
            return id.equals(s.id);
        }

        @Override
        public int hashCode() { return Objects.hash(id); }

        // 任务 3:实现接口的方法
        @Override
        public double score() { return examScore; }
    }

    // 任务 2:教师
    static class Teacher extends Person {
        private final String course;

        Teacher(String name, int age, String course) {
            super(name, age);
        }
    }

```

```

        this.course = course;
    }

    @Override
    public String toString() { return super.toString() + " 教 " + course; }

    @Override
    public double score() { return 40.0; }    // 课时考核分(演示)
}

// 任务 3:考核接口(能力契约)
interface Assessable {
    double score();
}

// 任务 6:静态计数器
private static int totalCreated = 0;

static <T extends Person> T register(T person) {
    totalCreated++;
    return person;
}

public static void main(String[] args) {
    List<Person> people = new ArrayList<>();
    people.add(register(new Student("2024001", "张三", 19, 88.5)));
    people.add(register(new Student("2024002", "李四", 20, 76.0)));
    people.add(register(new Student("2024003", "王五", 18, 93.0)));
    people.add(register(new Teacher("陈老师", 35, "数据结构")));
    people.add(register(new Teacher("刘老师", 41, "操作系统")));
    System.out.println("任务6 共登记 " + totalCreated + " 人");

    System.out.println("== 任务1-3 人员名单(多态:toString 各打各的) ==");
    for (Person p : people) {
        System.out.println(" " + p);
        if (p instanceof Assessable) {
            System.out.println("    考核分 = " + p.score());    // 动态绑定
        }
    }

    System.out.println("== 任务4 按学号去重 ==");
    Set<Student> students = new HashSet<>();
    students.add(new Student("2024001", "张三", 19, 88.5));
    students.add(new Student("2024001", "张山", 20, 90.0));    // 同号:应去重
    students.add(new Student("2024002", "李四", 20, 76.0));
    System.out.println("    去重后学生数: " + students.size() + " (应为 2)");

    System.out.println("== 任务5 按成绩降序前3名 ==");
    students.stream()
        .sorted((a, b) -> Double.compare(b.score(), a.score()))    // lambda 比较器
        .limit(3)
        .forEach(s -> System.out.println(" " + s + " 成绩 " + s.score()));

    System.out.println("== 任务7 报告完成 ==");
}
}

```

设计说明:任务 3 把 score() 做成接口方法,main 里统一按 Assessable 调用——新增“行政人员”类只需实现接口,统计代码零改动(开闭原则);任务 4 用“同号不同名”验证 equals 按学号判等,同时演示了为什么判等字段要 final;任务 5 的 Double.compare 比直接减号更安全(防 NaN 与溢出)。扩展方向:成绩从控制台输入(第3章)、用 Map<String, Student> 按学号索引(第6章)、异常处理(第9章)、record 简化 Person(第14章)。

下一章预告:第7章 文件与 I/O。字节流与字符流、Files/Path API、序列化与反序列化——把数据落盘,是程序离开内存的第一步。